ASCII character utilities

Document number:
P3688R2

Date:
2025-08-14

Audience:
SG16

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:
Jan Schultke <janschultke@gmail.com>

Co-Authors:
Corentin Jabot <corentin.jabot@gmail.com>

GitHub Issue:
wg21.link/P3688/github

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/ascii.cow

ABSTRACT: The utilities in `<cctype>` or `<locale>` are locale-specific, not `constexpr`, and provide no support for Unicode character types. We propose lightweight, locale-independent alternatives.

Contents

§ 1. Revision history

§ 1.1. Changes since R1

- In §5. Wording, fix incorrect return type for `ascii_case_insensitive_compare` in synopsis
- In §5. Wording, fix superfluous `std::` prefix for `strong_ordering` in definition of `ascii_case_insensitive_compare`

§ 1.2. Changes since R0

- In §3.3. `base` parameter in `is_ascii_digit`, explain why the precondition is not hardened
- In §5. Wording, fix a missing addition to [tab:headers.cpp]
- Minor editorial changes

§ 2. Introduction

Testing whether a character falls into a specific subset of ASCII characters or performing some simple transformations are common tasks in text processing. For example, applications may need to check if identifiers are comprised of alphanumeric ASCII characters or underscores; Unicode properties are not relevant to this task, and usually, neither are locales.

Unfortunately, these common and simple tasks are only supported through functions in the `<cctype>` and `<locale>` headers, such as:

```cpp
// <cctype>
int isalnum(int ch);
int isalpha(int ch);
// ...
int toupper(int ch);

// <locale>
template<class charT> bool isalnum(charT c, const locale& loc);
```

Especially the `<cctype>` functions are ridden with problems:

1. There is no support for Unicode character types (`char8_t`, `char16_t`, and `char32_t`).
2. These functions are not `constexpr`, but performing basic characters tests would be useful at compile time.
3. There are distinct function names for `char` and `wchar_t` such as `std::isalnum` and `std::iswalnum`, making generic programming more difficult.
4. If `char` is signed, these functions can easily result in undefined behavior because the input must be representable as `unsigned char` or be `EOF`. If `char` represents a UTF-8 code unit, passing any non-ASCII code unit into these functions has undefined behavior.
5. These functions violate the zero-overhead principle by also handling an `EOF` input, and in many use cases, `EOF` will never be passed into these functions anyway. The caller can easily deal with `EOF` themselves.
6. The return type of charater tests is `int`, where a nonzero return value indicates that a test succeeded. This is very unnatural in C++, where `bool` is more idiomatic.
7. Some functions use the currently installed `"C"` locale, which makes their use questionable for high-performance tasks because each invocation is typically an opaque call that checks the current locale.

We propose lightweight replacement functions which address all these problems.

> NOTE: Many of these problems are resolved by the `std::locale` overloads in `<locale>`, but their locale dependence makes them unfit for what this proposal aims to achieve.
>
> Testing whether a `char8_t` (assumed to be a UTF-8 code unit) is an ASCII digit is obviously a locale-independent task.

§ 2.1. Can't you implement this trivially yourself?

It is worth noting that some of the functions can be implemented very easily by the user. For example, existing code may already use a check like `c >= '0' && c <= '9'` to test for ASCII digits, and our proposed `is_ascii_digit` does just that.

However, not all of the proposed functions are this simple. For example, checking whether a `char` is an ASCII punctuation character (`'#'`, `'?'`, etc.) would require lots of separate checks

done naively. In the standard library, it can be efficiently implemented using a 128-bit or 256-bit bitset.

Even if all proposed functions were trivial to implement, working with ASCII characters is such an overwhelmingly common use case that it's worth supporting in the standard library.

3. Design

All proposed functions are `constexpr`, locale-independent, overloaded (i.e. no separate name for separate input types), and accept any character type (`char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t`). Furthermore, all function names contain `ascii` to raise awareness for the fact that these functions do not handle Unicode characters. A user would expect `is_upper(U'Ä')` to be `true`, but `is_ascii_upper(U'Ä')` to be `false`.

EXAMPLE: The counterpart to `std::isalpha` is declared follows:

```
constexpr bool is_ascii_alpha(character-type c) noexcept;
```

`character-type` means that there exists an overload set where this placeholder is replaced with each of the character types. This design is more consistent with `std::from_chars` and `<cmath>` functions than say, `template<class Char>`. Equivalent functions could also be added to C, if there is interest. This signature also allows the use with types that are convertible to a specific character type.

3.1. List of proposed functions

Find below a list of proposed functions. Note that the character set notation `[...]` is taken from RegEx.

`<cctype>`	Proposed name	Returns (given ASCII `char c`)
N/A	`is_ascii`	`c <= 0x7F`
`isdigit`	`is_ascii_digit`	`true` if `c` is in `[0-9]`, otherwise `false`
N/A	`is_ascii_bit`	`c == '0' || c == '1'`
N/A	`is_ascii_octal_digit`	`true` if `c` is in `[0-7]`, otherwise `false`
`isxdigit`	`is_ascii_hex_digit`	`true` if `c` is in `[0-9A-Fa-f]`, otherwise `false`
`islower`	`is_ascii_lower`	`true` if `c` is in `[a-z]`, otherwise `false`
`isupper`	`is_ascii_upper`	`true` if `c` is in `[A-Z]`, otherwise `false`
`isalpha`	`is_ascii_alpha`	`is_ascii_lower(c) || is_ascii_upper(c)`

`isalnum`	`is_ascii_alphanumeric`	`is_ascii_alpha(c) || is_asci_digit(c)`
`ispunct`	`is_ascii_punctuation`	`true` if `c` is in `[!"#$%&'()*+,\-./:;<=>?@\[\\]^_`{|}~]`, otherwise `false`
`isgraph`	`is_ascii_graphical`	`is_ascii_alphanumeric(c) || is_ascii_punctuation(c)`
`isprint`	`is_ascii_printable`	`is_ascii_graphical(c) || c == ' '`
`isblank`	`is_ascii_horizontal_whitespace`	`c == ' ' || c == '\t'`
`isspace`	`is_ascii_whitespace`	`true` if `c` is in `[ \f\n\r\t\v]`, otherwise `false`
`iscntrl`	`is_ascii_control`	`(c >= 0 && c <= 0x1F) || c == '\N{DELETE}'`
`tolower`	`ascii_to_lower`	the respective lower-case character if `is_ascii_upper(c)` is `true`, otherwise `c`
`toupper`	`ascii_to_upper`	the respective upper-case character if `is_ascii_lower(c)` is `true`, otherwise `c`
N/A	`ascii_case_insensitive_compare`	*see §3.5. Case-insensitive comparison functions*
N/A	`ascii_case_insensitive_equals`	*see §3.5. Case-insensitive comparison functions*

DECISION: The proposed names are mostly unabbreviated to fit the rest of the standard library style. Shorter names such as `is_ascii_alphanum` or `is_ascii_alnum` could also be used.

DECISION: `isgraph` should perhaps have no new version. It is of questionable use, and both the old and new name aren't obvious. In the default `"C"` locale, `isgraph` is simply `isprint` without `' '`.

Similarly, `isblank` should perhaps have no new version either. This proposal simply has a new version for every `<cctype>` function; if need be, they are easy to remove.

§ 3.2. `is_ascii`

This additional function is mainly useful for checking if a character "is ASCII", i.e. falls into the basic latin block, before performing an ASCII-only evaluation.

EXAMPLE: In the following overload set, the `char32_t` implementation delegates to the `char8_t` implementation to avoid repetition of its logic. The `std::is_ascii(c)` check is needed because because an unconditional `get_hex_digit_value(char8_t(c))`

may result in treating U+0130 LATIN CAPITAL LETTER I WITH DOT ABOVE as U+0030 DIGIT ZERO.

```
int get_hex_digit_value(char8_t c) {
    return c >= u8'0' && c <= u8'9' ? c - u8'0'
         : c >= u8'A' && c <= u8'F' ? c - u8'A'
         : c >= u8'a' && c <= u8'f' ? c - u8'a'
         : -1;
}

int get_hex_digit_value(char32_t c) {
    return std::is_ascii(c) ? get_hex_digit_value(char8_t(c)) : -1;
}
```

3.3. `base` parameter in `is_ascii_digit`

Similar to `std::to_chars`, `std::is_ascii_digit` can also take a `base` parameter:

```
constexpr bool is_ascii_digit(character-type c, int base = 10);
```

If `base` $\leq$ `10`, the range of valid ASCII digit character is simply limited. For greater `base`, a subset of alphabetic characters is also accepted, starting with `'a'` or `'A'`. Such a function is useful when parsing numbers with a base of choice, which is what `std::to_chars` does, for example.

Similar to `std::from_chars` and `std::to_chars`, the given `base` has to be between 2 and 36 (inclusive). This is a non-hardened precondition because all functions in `<ascii>` are low-level, high-performance, and spiritually numeric. Hardened preconditions are not used within that context.

3.4. `is_ascii_bit` and `is_ascii_octal_digit`

C++ and various other programming languages support binary and octal literals, so it seems like an arbitrary choice to only have dedicated overloads for (hexa)decimal digits. `is_ascii_bit` may be especially useful, such as when dealing with bit-strings like one of the `std::bitset` constructors.

In conclusion, we may as well have functions for bases 2, 8, 10, and 16; they're not doing much harm, they're trivial to implement, and some users may find them useful.

NOTE: None of the authors feel strongly about this, so if LEWG insists, we could remove `is_ascii_bit` and `is_ascii_octal_digit`, and even remove `is_ascii_hex_digit`, leaving only the multi-base `is_ascii_digit`.

3.5. Case-insensitive comparison functions

As shown in the table above, we also propose the case-insensitive comparison functions.

```
constexpr strong_ordering ascii_case_insensitive_compare(
    character-type a,
    character-type b
) {
    return ascii_to_upper(a) <=> ascii_to_upper(b);
}

constexpr strong_ordering ascii_case_insensitive_equals(
    character-type a,
    character-type b
) {
    return ascii_to_upper(a) == ascii_to_upper(b);
}
```

3.6. Why no function objects?

For case-insensitive comparisons and for character tests in general, function objects may be convenient because they can be more easily used in algorithms:

```
std::string_view str = "abc123";
// This does not work if is_ascii_digit is an overloaded function or function t
auto it = std::ranges::find(str, is_ascii_digit);
```

However, there is no reason why `is_ascii_digit` *needs* to be a function object. It is not a customization point, but a plain function. Furthermore, defining function objects for this purpose may be obsoleted by [P3312R1] Overload Set Types.

3.7. What to do for ASCII-incompatible `char` and `wchar_t`

Not every ordinary and wide character encoding is ASCII-compatible, such as EBCDIC, Shift-JIS, and (defunct) ISO-646, i.e. code units ≤ `0x7f` do not represent the same characters as ASCII.

This begs the question: what should `is_ascii_digit('0')` do on an EBCDIC platform, where this call is `is_ascii_digit(char(0xf0))` ? We have three options, discussed below.

> NOTE: `is_ascii_digit(u8'0')` is equivalent to `is_ascii_digit(char8_t(0x30))` on any platform. In general, the behavior for Unicode character types is obvious, unlike that for `char` and `wchar_t`.

3.7.1. Conditionally supported `char` overloads

We could mandate that the ordinary literal encoding is an ASCII superset for the `char` overload to exist. This would force a cast (to `char8_t`) to use the functions on EBCDIC platforms. It is not

clear how implementations would treat Shift-JIS; GCC assumes `'\\' == '¥'` to be `true`, so this option may not be enough to alleviate the awkwardness of `is_ascii_punctuation('¥')`.

Also, this option is not very useful. It is reasonable to have UTF-8 data stored in a `char[]` on EBCDIC platforms, and having to perform casts to `char8_t` would be awkward.

3.7.2. Transcode `char` to ASCII

We could transcode from the ordinary literal encoding to ASCII and produce an answer for the result of that transcoding. This would be a greater burden for implementations, especially on EBCDIC platforms. The benefit is that `is_ascii_digit('0')` is always `true`, although `is_ascii_digit(char(0x30))` may not be. However, `is_ascii_digit(char8_t(0x30))` is always `true`.

It probably does not solve the `is_ascii_punctuation('¥')` case, as implementers may keep transcoding `'¥'` and `'\\'` in the same way. It would also give incorrect answers for stateful encodings. There are EBCDIC control characters that do not have an ASCII equivalent, so if we were to do conversions, we would have to decide what, for example, `is_ascii_control('\u008B')` should produce.

> NOTE: This option was originally preferred by one of the authors, but proved to be *hugely* unpopular in discussion of the proposal.

3.7.3. Treat the input as ASCII, regardless of the literal encoding

This is our proposed behavior.

The most simple option is to ignore literal encoding entirely, and assume that `char` inputs are ASCII-encoded. The greatest downside is that depending on encoding, `is_ascii_digit('0')` may be `false`, which may be surprising to the user. However, the main purpose of these functions is to be called with characters taken from ASCII text, so what results they yield when passing literals is not so important.

There are use cases for this behavior on EBCDIC platforms. A lot of protocols (HTTP, POP) and file formats (JSON, XML) are ASCII/UTF-8-based and need to be supported on EBCDIC systems, making these functions universally useful, especially as `<cctype>` functions cannot easily be used to deal with ASCII on these platforms.

Ultimately, do we want functions to deal with ASCII or the literal encoding? If we want them to be a general way to query the ordinary literal encoding, `is_ascii` is a terrible name, and finding a more general name would prove difficult.

> NOTE: If we choose this option, we can still provide the same transcoding functionality as the previous option by offering a (literal-encoded) `char` → (code point) `char32_t` function, although that may be outside the scope of this proposal.

3.8. What if the input is a non-ASCII code unit?

Text input is rarely guaranteed to be pure ASCII, i.e. some code units may be > `0x7f`. However, we're still interested in ASCII characters within that input. For example, we may

- parse pure ASCII numbers like `123` in a UTF-8 JSON (or other config) file,
- trim ASCII whitespace in HTTP headers, which are encoded with ISO-8859-1,
- parse ASCII-alphanumeric variable names in Lua scripts, where non-ASCII characters can appear (comments, string),
- ...

It is possible (and expected) that the user calls say, `is_ascii_digit(U'ö')`, at least indirectly. For the sake of convenience, all proposed functions should handle such inputs by

- returning `false` in the case of all testing functions, and
- applying an identity transformation in transformation/case-insensitive comparison functions.

EXAMPLE: With these semantics, the user can safely write:

```
std::u8string_view str = u8"öab 123";
// it is an iterator to '1' because 'ö' is skipped
auto it = std::ranges::find(str, [](char8_t c) { return std::is_ascii_dig
```

If `is_ascii_digit` doesn't simply return `false` on non-ASCII inputs, the proposal is useless for the common use case where some non-ASCII characters exist in the input.

The proposed behavior also works excellently with any ASCII-compatible encoding, such as UTF-8. Surrogate code units in UTF-8 are all greater than `0x7F`, so if we implement say, `is_ascii_digit` naively by checking `c >= '0' && c <= '9'`, it "just works".

§ 3.9. Why not accept any integer type?

Some people argue that a test like `is_ascii_digit('0')` is a purely numerical test using the ASCII table, and so passing `is_ascii_digit(0x30)` should also be valid.

However, this permissive interface would invite bugs. For example, `c - '0'` is the difference between ASCII characters, not an ASCII character, so passing it into `is_ascii_digit` would be nonsensical. Static type systems exist for a reason: to protect us from stupid mistakes. While `char`, `char32_t` etc. are not required to be ASCII-encoded, they are at least characters, so passing them into our functions is likely something the user intended to do, which we cannot say with confidence about `int`, `unsigned int`, etc.

Additionally, if we allowed passing signed integers, we may want to make the behavior erroneous or undefined for negative inputs because `is_ascii_digit(-1'000'000)` is most likely a developer mistake. Our interface is very simple: it has a wide contract and almost all functions are `noexcept`. Let's keep it that way!

Lastly, even proponents of passing integer types would not want `is_ascii_digit(true)` to be valid.

§ 3.10. ASCII case-insensitive views and case transformation algorithms

Ignoring or transforming ASCII case in algorithms is a fairly common problem. Therefore, it may be useful to provide views such as `std::views::ascii_lower`, algorithms like `std::ranges::equal_ascii_case_insensitive`, etc.

EXAMPLE: HTML tag names are case-insensitive and comprised of ASCII characters, like `<div>`, `<DIV>` etc. To identify a `<div>` element, it would be nice if the user could write:

```
std::ranges::equal(tag_name | std::views::ascii_lower, "div");
// or
std::ranges::ascii_case_insensitive_equal(tag_name, "div");
// or
tag_name.ascii_case_insensitive_equals("div");
```

While case transformations can be implemented naively using `std::transform`, dedicated functions would allow an efficient vectorized implementation for contiguous ranges, which can be many times faster ([AvoidCharByChar], [AVX-512CaseConv]) Similarly, a case-insensitive comparison function can be vectorized. In fact, POSIX's `strncasecmp` has been heavily optimized in glibc ([AVX2strncasecmp]), and providing range-based interfaces would allow delegating to these heavily optimized functions.

We intend to propose such utilities in a future paper or revision of this paper. Currently, this proposal is focused exclusively on operations involving character types.

§ 3.11. Why just ASCII?

It may be tempting to generalize the proposed utilities beyond ASCII, e.g. to UTF-8. However, this is not proposed for multiple reasons:

- You cannot pass `char8_t` into a UTF-8 `is_upper` function and expect meaningful results. In general, operations on variable-length encodings require sequences of code units. The interface we propose *only* makes sense for ASCII.
- Unicode utilities are tremendously more complex than ASCII utilities. Some Unicode case conversions even require multi-code-point changes.

§ 4. Implementation experience

A naive implementation of all proposed functions can be found at [CompilerExplorer], although these are implemented as function templates, not as overload sets (as proposed).

A more advanced implementation of some functions can be found in [µlight]. Character tests can be optimized using 128-bit or 256-bit bitsets.

§ 5. Wording

The wording changes are relative to [N5008].

In [tab:headers.cpp], add a new element to C++ library headers table:

```
<ascii>
```

In subclause [version.syn], update the synopsis as follows:

```
[...]
#define __cpp_lib_as_const                          201510L // freestandi
#define __cpp_lib_ascii                             20XXXXL // freestandi
#define __cpp_lib_associative_heterogeneous_erasure 202110L // also in [...]
[...]
```

In Clause [text], append a new subclause:

ASCII utilities [ascii]

Subclause [ascii] describes components for dealing with characters that are encoded using ASCII or encodings that are ASCII-compatible, such as UTF-8.

Recommended practice: Implementations should emit a warning when a function in this subclause is invoked using a value produced by a *string-literal* or *character-literal* whose encoding is ASCII-incompatible.
[*Example*: `is_ascii_digit('0')` is `false` if the ordinary literal encoding ([lex.charset]) is EBCDIC or some other ASCII-incompatible encoding, which can be surprising to the user. However, `is_ascii_digit(char{0x30})` is `true` regardless of ordinary literal encoding. — *end example*]

Header <ascii> synopsis [ascii.syn]

When a function is specified with a type placeholder of *character-type*, the implementation provides overloads for all character types ([basic.fundamental]) in lieu of *character-type*.

```
// all freestanding
namespace std {
  // [ascii.chars.test], ASCII character testing
  constexpr bool is_ascii(character-type c) noexcept;
```

```
  constexpr bool is_ascii_digit(character-type c, int base = 10);
  constexpr bool is_ascii_bit(character-type c) noexcept;
  constexpr bool is_ascii_octal_digit(character-type c) noexcept;
  constexpr bool is_ascii_hex_digit(character-type c) noexcept;

  constexpr bool is_ascii_lower(character-type c) noexcept;
  constexpr bool is_ascii_upper(character-type c) noexcept;
  constexpr bool is_ascii_alpha(character-type c) noexcept;
  constexpr bool is_ascii_alphanumeric(character-type c) noexcept;

  constexpr bool is_ascii_punctuation(character-type c) noexcept;
  constexpr bool is_ascii_graphical(character-type c) noexcept;
  constexpr bool is_ascii_printable(character-type c) noexcept;

  constexpr bool is_ascii_horizontal_whitespace(character-type c) noexcept
  constexpr bool is_ascii_whitespace(character-type c) noexcept;

  constexpr bool is_ascii_control(character-type c) noexcept;

  // [ascii.chars.transform], ASCII character transformation
  constexpr character-type ascii_to_lower(character-type c) noexcept;
  constexpr character-type ascii_to_upper(character-type c) noexcept;

  // [ascii.chars.case.compare], ASCII case-insensitive character comparison
  constexpr strong_ordering ascii_case_insensitive_compare(character-type
                                                           character-type
  constexpr bool ascii_case_insensitive_equals(character-type a
                                               character-type b) noexcept;
}
```

ASCII character testing [ascii.chars.test]

```
constexpr bool is_ascii(character-type c) noexcept;
```

Returns: `static_cast<char32_t>(c) <= 0x7F`.

```
constexpr bool is_ascii_digit(character-type c, int base = 10);
```

Preconditions: `base` has a value between 2 and 36 (inclusive).

Returns:

```
   (static_cast<char32_t>(c) >= U'0' && static_cast<char32_t>(c) < U'
|| (static_cast<char32_t>(c) >= U'a' && static_cast<char32_t>(c) < U'
|| (static_cast<char32_t>(c) >= U'A' && static_cast<char32_t>(c) < U'
```

Remarks: A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression.

```
constexpr bool is_ascii_bit(character-type c) noexcept;
```

Returns: is_ascii_digit(c, 2).

```
constexpr bool is_ascii_octal_digit(character-type c) noexcept;
```

Returns: is_ascii_digit(c, 8).

```
constexpr bool is_ascii_hex_digit(character-type c) noexcept;
```

Returns: is_ascii_digit(c, 16).

```
constexpr bool is_ascii_lower(character-type c) noexcept;
```

Returns: static_cast<char32_t>(c) >= U'a' && static_cast<char32_t>(c) <= U'z'.

```
constexpr bool is_ascii_upper(character-type c) noexcept;
```

Returns: static_cast<char32_t>(c) >= U'A' && static_cast<char32_t>(c) <= U'Z'.

```
constexpr bool is_ascii_alpha(character-type c) noexcept;
```

Returns: is_ascii_lower(c) || is_ascii_upper(c).

```
constexpr bool is_ascii_alphanumeric(character-type c) noexcept;
```

Returns: is_ascii_alpha(c) || is_ascii_digit(c).

```
constexpr bool is_ascii_punctuation(character-type c) noexcept;
```

Returns: u32string_view(U"!\"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~")
.contains(static_cast<char32_t>(c)).

```
constexpr bool is_ascii_graphical(character-type c) noexcept;
```

Returns: is_ascii_alphanumeric(c) || is_ascii_punctuation(c).

```
constexpr bool is_ascii_printable(character-type c) noexcept;
```

Returns: is_ascii_graphical(c) || static_cast<char32_t>(c) == U' '.

```
constexpr bool is_ascii_horizontal_whitespace(character-type c) noexcept;
```

Returns: static_cast<char32_t>(c) == U' ' || static_cast<char32_t>(c) == U'\t'.

```
constexpr bool is_ascii_whitespace(character-type c) noexcept;
```

Returns: u32string_view(U"\f\n\r\t\v").contains(static_cast<char32_t>(c)).

```
constexpr bool is_ascii_control(character-type c) noexcept;
```

Returns: static_cast<char32_t>(c) <= 0x1F || static_cast<char32_t>(c) == U'\N{DELETE}'.

ASCII character transformation [ascii.chars.transform]

```
constexpr character-type ascii_to_lower(character-type c) noexcept;
```

Returns: `is_ascii_upper(c) ? static_cast<character-type>(static_cast<char32_t>(c) - U'A' + U'a') : c`.

```
constexpr character-type ascii_to_upper(character-type c) noexcept;
```

Returns: `is_ascii_lower(c) ? static_cast<character-type>(static_cast<char32_t>(c) - U'a' + U'A') : c`.

ASCII case-insensitive character comparison [ascii.chars.case.compare]

```
constexpr strong_ordering ascii_case_insensitive_compare(character-type a,
                                                         character-type b);
```

Returns: `ascii_to_upper(a) <=> ascii_to_upper(b)`.

```
constexpr bool ascii_case_insensitive_equals(character-type a,
                                             character-type b) noexcept;
```

Returns: `ascii_to_upper(a) == ascii_to_upper(b)`.

NOTE: Some uses of `static_cast` are unnecessary to describe semantics. For example, `static_cast<char32_t>(c) == U' '` is equivalent to `c == U' '`.

However, these uses of `static_cast` may improve readability and avoid the use of behavior which is proposed to be deprecated in [P3695R0].

§ 6. References

[N5008] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-03-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf

[P3312R1] *Bengt Gustafsson.* Overload Set Types (2025-04-16)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3312r1.pdf

[P3695R0] *Jan Schultke.* Deprecate implicit conversions between char8_t, char16_t, and char32_t (2025-05-18)
https://isocpp.org/files/papers/P3695R0.html

[CompilerExplorer] *Jan Schultke, Corentin Jabot.* Partial implementation of character utilities
https://godbolt.org/z/5nvWzdf8G

[µlight] *Jan Schultke.* ascii_chars.hpp utilities in µlight
https://github.com/Eisenwave/ulight/blob/main/include/ulight/impl/ascii_chars.hpp

[AVX2strncasecmp] *Noah Goldstein.* glibc [PATCH v1 21/23] x86: Add AVX2 optimized str{n}casecmp (2022-03-23)
https://sourceware.org/pipermail/libc-alpha/2022-March/137272.html

[AvoidCharByChar] *Daniel Lemire.* Avoid character-by-character processing when performance matters (2020-07-21)
https://lemire.me/blog/2020/07/21/avoid-character-by-character-processing-when-performance-matters/

[AVX-512CaseConv] *Daniel Lemire.* Converting ASCII strings to lower case at crazy speeds with AVX-512 (2024-08-03)
https://lemire.me/blog/2024/08/03/converting-ascii-strings-to-lower-case-at-crazy-speeds-with-avx-512/